PROJECT 02 — WEB APPLICATION SECURITY

Campus Portal

A secure authentication and cryptography system

DURATION	PLATFORM	FOCUS	STATUS
6 weeks	Python · SQLite	Auth · access · crypto	Planning

STARTING POINT

1

Technical requirements

Web application with user and administrator roles

Registration, login, logout and secure sessions

Password hashing and account control

Server-side authorisation and access control

Encryption plus an HMAC or signature demonstration

2

Tools and technologies

Python / Flask, PHP or Node.js

SQLite, MySQL or MariaDB

Burp Suite

OpenSSL

Browser developer tools

3

Expected deliverables

Source code and database

Architecture diagram

Security test cases

Report and live demonstration

SUCCESS CRITERION

The solution must be functional, tested, documented and presented with technical evidence.

OBJECTIVE

Building a Secure App

A student management portal that handles authentication, authorisation and cryptography correctly. Correct behaviour on its own is not evidence, so the project will also show what each protection actually prevents.

01

Build the product

A working portal with real accounts, courses, grades, certifications and private messaging.

02

Build it twice

A baseline version carrying the classic flaws, and a hardened version that closes each one.

03

Prove the difference

The same request sent to both versions, with the outcome recorded as a security test case.

APPROACH

One codebase, two builds

BASELINE

Baseline build

The version that gets attacked

— Carries the classic flaws on purpose: injection, forged sessions, missing checks

— Passwords kept with a fast, unsalted digest

— Messages and diplomas stored with no protection at all

— Runs on its own port, for the attack half of the demonstration

HARDENED

Hardened build

The version that holds

— Every value bound to the query as data, never assembled into it

— Sessions verified before anything in them is believed

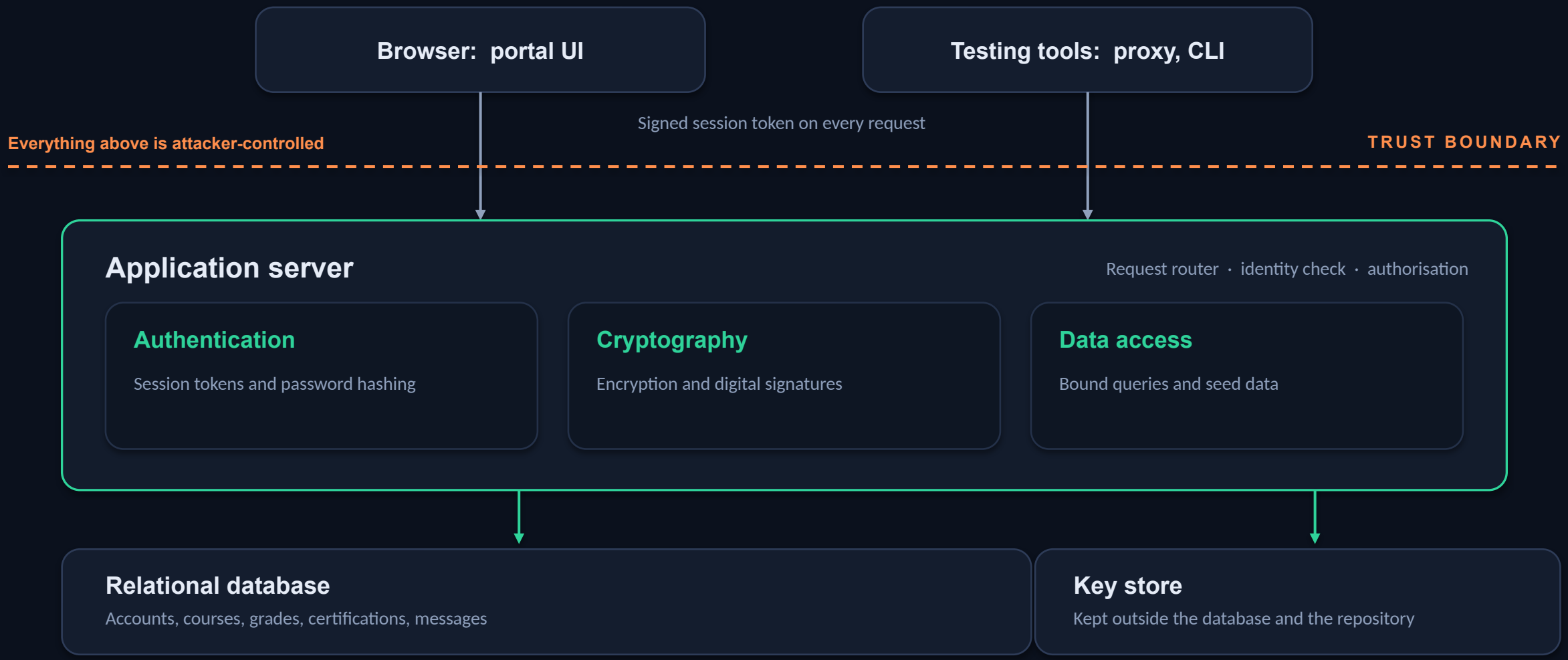
— Every privileged action re-checked against the database

— Runs alongside the baseline, for the proof half of the demonstration

A single environment setting selects the build, so both can run side by side during the presentation.

DESIGN

System architecture



ACTORS

The three roles in the system

S

Student

Consumes their own record

— View enrolled courses and the grades received

— View certifications issued to them

— Private messaging with staff and peers

P

Professor

Owns a set of courses

— View their own courses and the students enrolled

— Enter and update grades for those courses only

— Private messaging

A

Administrator

Governs the institution

— Create and manage every account

— Issue and sign certifications

— Full visibility across the portal

HOW THE ROLE IS USED

The role lives on the account record. It is re-read from the database for every privileged action, and never taken from the incoming request.

ACCESS CONTROL

What each role is allowed to do

Action	Student	Professor	Admin	Rule the server applies
View own courses and grades	Yes	Yes	Yes	Verified identity from the session
Enter a grade	No	Own courses	Yes	The course must belong to them
Issue a certification	No	No	Yes	Administrator role, read from the database
Open a conversation	Own only	Own only	Yes	Must be one of the two participants
List every account	No	No	Yes	Admin role; credentials never returned
Choose one's own role	No	No	No	The server assigns the role at registration

INFORMATION

What the system stores

Accounts

Identity, credentials and the role that drives every authorisation decision.

Courses

The catalogue, each course owned by exactly one professor.

Enrolments

Which student follows which course.

Grades

One mark per student per course, with the author of the change recorded.

Certifications

Diplomas issued by the administration, each carrying a digital signature.

Messages

Private one-to-one conversations, sealed before they reach storage.

Six entities, one database per build, so the two versions never share state.

SCOPE

Six security pillars to implement

01 Authentication

Sessions carried by a signed token whose signature and expiry are checked on every request.

02 Password storage

A deliberately slow, salted key-derivation function with constant-time comparison.

03 Authorisation

Every privileged action re-reads the caller's role and ownership from the database.

04 Input handling

Values are bound to the query as data; user input never becomes part of the statement.

05 Encryption at rest

Message bodies sealed with authenticated symmetric encryption before storage.

06 Digital signatures

Each certification signed by the issuer, with the public key published for independent checks.

SCHEDULE

Six weeks, six phases

W1 — W2 — W3 — W4 — W5 — W6

Design

- Scenario and objectives
- Architecture
- Project setup

Implement

- Accounts and sign-in
- Courses and grades
- Private messaging

Secure

- Verified sessions
- Server-side access control
- Encryption and signatures

Test

- Attack scenarios
- Validation runs
- Test cases recorded

Improve

- Analysis and remediation
- Gaps corrected
- Optimisation

Deliver

- Evidence and report
- Architecture diagram
- Live demonstration

PRINCIPLE

Each week produces a concrete result; week 6 turns the work into a demonstrable, documented project.

OUTPUT

What will be handed in

1

Source code and database

The portal in both builds, with the schema and reproducible seed data.

2

Architecture diagram

The system map, the trust boundary and the data model.

3

Security test cases

Every attack with its expected result on each build.

4

Report and demonstration

The written analysis plus a live run in front of the class.

SUCCESS CRITERION

The solution must be functional, tested, documented and presented with technical evidence. Every claim in the report will be backed by a request anyone in the room can replay.

IN CLOSING

The security model in four rules

01

Input is data, never code

Values are bound to a query; they never write it.

02

A token is worth nothing unverified

Check the signature and the expiry before believing it.

03

Authorisation starts from server state

Never from a claim the client sends, however well formed.

04

Encryption and access control pair up

One protects the file, the other protects the interface.

Next step Week 2: Implementation